Data Structures (ETCCDS202)

Unit–1 Assignment: Foundations & Algorithmic Analysis

Name of the School: School of Engineering & Technology

Program: B.Tech CSE AIML SEC-A

Course Title: Data Structures

Course Code: ETCCDS202

Unit Title: Foundations & Algorithmic Analysis

Student Name: OMESH VERMA

Roll No.:2501730331

Section: A

Submitted to: Mrs.Neetu Chauhan

Submission Date: 5/03/2026

1. Understanding ADT (Abstract Data Type)

An Abstract Data Type, or ADT, is a way of describing a data structure based on the operations it supports rather than how it is implemented internally. In simple words, ADT focuses on “what a data structure does” instead of “how it does it”. For example, a Stack ADT allows operations like push, pop, peek, size, and checking if the stack is empty. The internal implementation can use arrays, lists, or other structures, but the behaviour remains the same.

Stack operations such as push and pop usually take constant time O(1). This is because these operations only affect the top element of the stack. No searching or traversal is needed, so the number of steps does not grow with the size of the stack.

2. Recursive Factorial

The factorial of a number n is calculated as $n \times (n-1) \times (n-2) \ldots \times 1$. In recursion, the function calls itself with a smaller value until it reaches a base case. The base case used here is $0! = 1$.

Time Complexity:

The factorial function makes one recursive call for each number from n down to 0. Because of this, the time complexity is O(n).

Space Complexity:

Since each recursive call is stored in the call stack until the computation finishes, the recursion depth is also n. Therefore, the space complexity is O(n).

In this algorithm, the best case, worst case, and average case are all the same because the function always performs the same number of recursive calls for a given n.

3. Fibonacci Numbers

The Fibonacci sequence is defined as:

$F(n) = F(n-1) + F(n-2)$

Two versions were implemented in this toolkit: a naive recursive approach and a memorized recursive approach.

Naive Fibonacci:

The naive recursive solution repeatedly calculates the same values again and again. For example, when calculating F(5), the function will calculate F(3) multiple times. Because of this repeated work, the time complexity becomes exponential.

Time Complexity: $O(2^n)$

Space Complexity: $O(n)$ due to recursion depth.

Memoized Fibonacci:

To improve efficiency, memoization stores results that have already been computed. When the function needs a value again, it simply retrieves it instead of recalculating it.

Time Complexity: $O(n)$

Space Complexity: $O(n)$ for storing results in memory.

The call counter in the program clearly shows that memoization drastically reduces the number of recursive calls, especially for larger values like 20 or 30.

4. Tower of Hanoi

The Tower of Hanoi is a classic recursion problem involving three rods and multiple disks. The goal is to move all disks from the source rod to the destination rod while following two rules: only one disk can be moved at a time, and a larger disk cannot be placed on top of a smaller disk.

The recursive idea works by:

1. Moving n−1 disks to the auxiliary rod.

2. Moving the largest disk to the destination rod.

3. Moving the n−1 disks from auxiliary to destination.

The number of moves required follows the recurrence:

$T(n) = 2T(n-1) + 1$

Solving this recurrence gives:

Total moves = $2^n - 1$

Time Complexity: $O(2^n)$

Space Complexity: $O(n)$ because the maximum recursion depth equals the number of disks.

5. Recursive Binary Search

Binary search works on a sorted array and repeatedly divides the search space into two halves.

Instead of checking every element like linear search, binary search only looks at the middle element

and decides which half to search next.

The recurrence relation for binary search is:

$T(n) = T(n/2) + O(1)$

Each recursive step cuts the problem size in half, so the time complexity becomes $O(\log n)$.

Best Case:

The element is found at the middle on the first check, giving $O(1)$.

Average Case:

Usually around $O(\log n)$.

Worst Case:

Still $O(\log n)$, because the array size keeps halving each step.

6. Algorithm Paradigms

Divide and Conquer:

Binary search is a good example of the divide and conquer approach. The problem is divided into smaller parts (halves of the array), and the algorithm focuses only on the relevant half.

Dynamic Programming:

Memoized Fibonacci demonstrates dynamic programming. Instead of solving the same subproblem multiple times, previously computed values are stored and reused. This avoids unnecessary computation.

Recursion:

Tower of Hanoi uses recursion because the problem can be broken into smaller versions of itself.

The base case occurs when only one disk needs to be moved.

7. Real-Life Examples of Paradigms

Divide and Conquer:

Searching for a word in a dictionary by opening near the middle and deciding whether to go left or right.

Dynamic Programming: Navigation systems such as Google Maps calculating the shortest route while storing intermediate results.

Greedy Algorithm: Giving change using the largest possible coin first.For example, using a 10 coin before 5 coins when making change.